Vanilla JavaScript, HTML and CSS

🔷 1. Build a Modal Popup

Prompt:

Create a modal dialog box using HTML, CSS, and JavaScript. It should:

- Open when a button is clicked.
- Close when the close icon (×) or overlay is clicked.

> Follow-up: Add keyboard accessibility to close modal on Escape.

solution → we have a model div in html , which on button click will call a function and set display style from none to block

we also attach an event listener to change CSS property to none when clicked on esc button

```
<html>
  <header>
    <title>Modal</title>
    <link rel="stylesheet" href="Modal.css" />
  </header>
  <body>
    <h1>Modal Popup</h1>
    <p>This is a simple modal popup example.</p>
    <button id="openModal" onclick="">Open Modal</button>

    <!-- Modal structure here -->
```

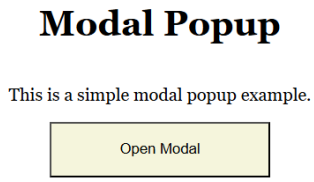
```
    <div id="myModal" class="modal">
      <div class="modal-content">
        <div class="modal-header">
          <h2>Modal Header</h2>
          <span id="closeModal" class="close">&times;</span>
        </div>
        <p>This is the modal content!</p>
      </div>
    </div>
  </body>
  <script src="Modal.js"></script>
</html>
```

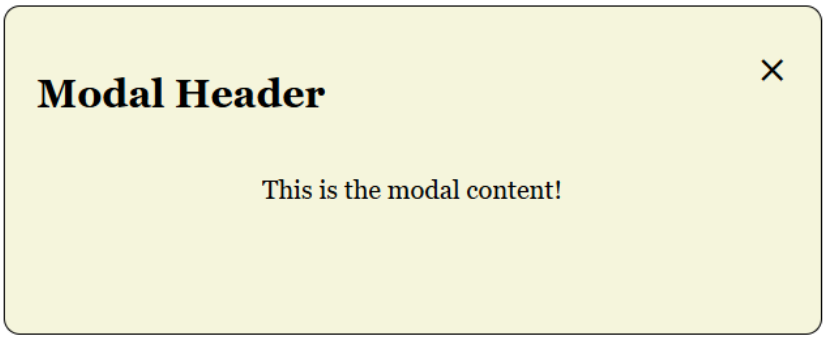
```
#openModal {
  width: 200px;
  height: 50px;
  background-color: beige;
  color: black;
  cursor: pointer;
}

body {
  font-family: Georgia, "Times New Roman", Times, serif;
  justify-content: center;
  display: flex;
  flex-direction: column;
  align-items: center;
  position: relative;
  height: 100vh;
  margin: 0;
  text-align: center;
}

.modal {
```

```
  display: none;
  position: fixed;
  z-index: 1;
  height: 200px;
  width: 500px;
  background-color: beige;
  border: 1px solid black;
  border-radius: 10px;
}

.modal-content {
  padding: 20px;
}

.modal-header {
  display: flex;
  justify-content: space-between;
}

.close {
  cursor: pointer;
  color: black;
  font-size: 30px;
}
```

```
const modal = document.getElementById("myModal");
const openBtn = document.getElementById("openModal");
const closeBtn = document.getElementById("closeModal");

openBtn.onclick = function () {
  modal.style.display = "block";
};
closeBtn.onclick = function () {
  modal.style.display = "none";
```

```
};

window.onclick = function (event) {
  if (event.target === modal) {
    modal.style.display = "none";
  }
};
// Close the modal when the Escape key is pressed
document.addEventListener("keydown", function (event) {
  if (event.key === "Escape") {
    modal.style.display = "none";
  }
});
```





2. Create a Todo List App

Prompt:

Build a simple Todo app with the following features:

- Input box to add a task.
- List to display tasks.
- Buttons to delete or mark as completed.
- Bonus: Persist todos using `localStorage`.

> Follow-up: Add a filter (All / Completed / Active).

```
<html>
  <header>
    <title>To-Do List</title>
```

```
    <link rel="stylesheet" href="Todo.css" />
  </header>
  <body>
    <div class="todo-container">
      <h1>Todo List</h1>
      <input type="text" id="todoInput" placeholder="Add a task..." />
      <button id="addBtn">Add</button>

      <div class="filters">
        <button data-filter="all" class="filter active">All</button>
        <button data-filter="active" class="filter">Active</button>
        <button data-filter="completed" class="filter">Completed</button>
      </div>

      <ul id="todoList"></ul>
    </div>
  </body>
  <script src="Todo.js"></script>
</html>
```

```
// Get reference to the input field where user types the todo
const todoInput = document.getElementById("todoInput");

// Get reference to the "Add" button
const addBtn = document.getElementById("addBtn");

// Get reference to the <ul> element where todos will be listed
const todoList = document.getElementById("todoList");

// Get all filter buttons (All / Active / Completed)
const filterBtns = document.querySelectorAll(".filter");

// Load existing todos from localStorage (or start with an empty array)
let todos = JSON.parse(localStorage.getItem("todos")) || [];
```

```
// Current selected filter (default is 'all')
let currentFilter = "all";

// Save the todos array into localStorage
function saveTodos() {
  localStorage.setItem("todos", JSON.stringify(todos));
}

// Render the todos on the screen based on current filter
function renderTodos() {
  // Clear all current list items before re-rendering
  todoList.innerHTML = "";

  // Filter todos based on selected filter
  const filtered = todos.filter((todo) => {
    if (currentFilter === "active") return !todo.completed; // Show only uncomplete
    if (currentFilter === "completed") return todo.completed; // Show only comple
    return true; // If filter is 'all', show everything
  });

  // Loop through each filtered todo and create list elements
  filtered.forEach((todo, index) => {
    // Create list item
    const li = document.createElement("li");

    // Create span element for todo text
    const text = document.createElement("span");
    text.textContent = todo.text;

    // If todo is marked as completed, add CSS class for styling
    if (todo.completed) text.classList.add("completed");

    // Toggle completed state when user clicks the text
    text.onclick = () => {
      todos[index].completed = !todos[index].completed; // Toggle
```

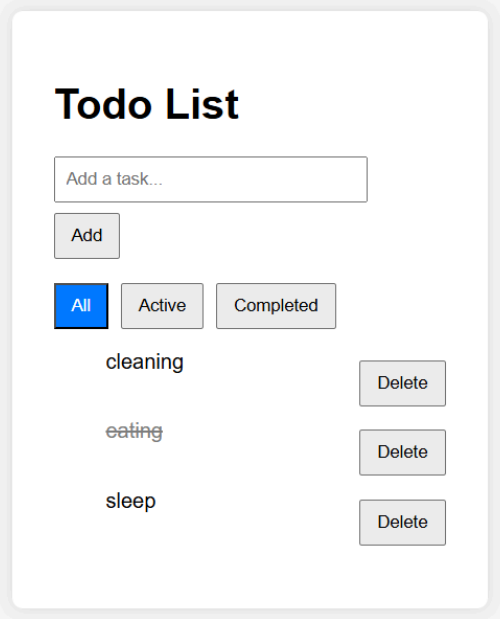
```
      saveTodos(); // Save updated list to localStorage
      renderTodos(); // Re-render updated UI
    };

    // Create "Delete" button
    const delBtn = document.createElement("button");
    delBtn.textContent = "Delete";

    // When delete button is clicked, remove the todo from the list
    delBtn.onclick = () => {
      todos.splice(index, 1); // Remove one element at current index
      saveTodos(); // Save updated list
      renderTodos(); // Re-render updated UI
    };

    // Add text and delete button to the list item
    li.appendChild(text);
    li.appendChild(delBtn);

    // Add the list item to the <ul>
    todoList.appendChild(li);
  });
}

// Handle "Add" button click to add a new todo
addBtn.onclick = () => {
  const value = todoInput.value.trim(); // Get input value and remove whitespace
  if (value !== "") {
    // Add new todo to the array
    todos.push({ text: value, completed: false });
    saveTodos(); // Save to localStorage
    renderTodos(); // Re-render UI
    todoInput.value = ""; // Clear input field
  }
};
```

```
// Loop through each filter button and attach click handlers
filterBtns.forEach((btn) => {
  btn.onclick = () => {
    // Remove 'active' class from all filter buttons
    filterBtns.forEach((b) => b.classList.remove("active"));

    // Add 'active' class to the clicked button
    btn.classList.add("active");

    // Update the current filter based on button's data-filter attribute
    currentFilter = btn.dataset.filter;

    // Re-render todos based on new filter
    renderTodos();
  };
});

// Call this once when page loads to show saved todos
renderTodos();
```

```
body {
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}

.todo-container {
  background: white;
  padding: 2rem;
  border-radius: 8px;
```

```
  width: 300px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

input[type="text"] {
  width: calc(100% - 60px);
  padding: 8px;
  margin-right: 8px;
}

button {
  padding: 8px 12px;
  margin-top: 8px;
  cursor: pointer;
}

.filters {
  margin: 10px 0;
}

.filters .filter {
  margin-right: 5px;
}

.filter.active {
  background-color: #007bff;
  color: white;
}

li {
  list-style: none;
  margin: 10px 0;
  display: flex;
  justify-content: space-between;
}
```

```css
.completed {
  text-decoration: line-through;
  color: grey;
}
```



🔷 3. Accordion or FAQ Section

Prompt:

Build an FAQ component. Clicking on a question expands/collapses the answer. Only one answer should be visible at a time.

> Follow-up: Implement it in a way that is reusable (use classes/data attributes).

```
<html>
 <header>
  <title>Accordion Example</title>
  <link rel="stylesheet" href="Accordion.css" />
 </header>
 <body>
  <div id="Accordion">
    <div class="accordion">
     <h3 class="title">FAQ section</h3>

     <div class="faq">
    <div class="question">
     <h3>What is Lorem Ipsum?</h3>
     <svg width="15" height="10" viewBox="0 0 42 25" fill="none">
      <path d="M3 3L21 21L39 3" stroke="black" stroke-width="7" stroke-linec
     </svg>
    </div>
     <div class="answer">
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer nec o
     </div>
     </div>

       <div class="faq">
    <div class="question">
     <h3>Why do we use it?</h3>
     <svg width="15" height="10" viewBox="0 0 42 25" fill="none">
      <path d="M3 3L21 21L39 3" stroke="black" stroke-width="7" stroke-linec
     </svg>
    </div>
     <div class="answer">
      <p>It is a long established fact that a reader will be distracted by the reac
     </div>
     </div>
```

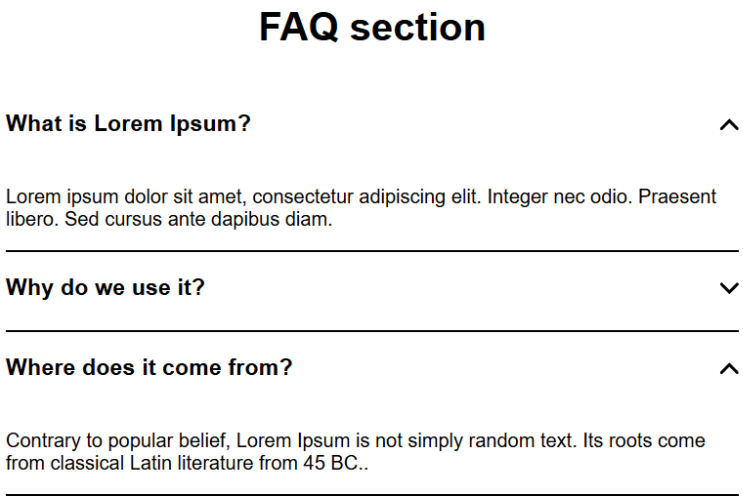
```
            <div class="faq">
        <div class="question">
          <h3>Where does it come from?</h3>
          <svg width="15" height="10" viewBox="0 0 42 25" fill="none">
            <path d="M3 3L21 21L39 3" stroke="black" stroke-width="7" stroke-linec
          </svg>
        </div>
          <div class="answer">
            <p>Contrary to popular belief, Lorem Ipsum is not simply random text. Its
          </div>
          </div>

    </div>
  </body>
  <script src="Accordion.js"></script>
</html>
```

```
body {
  font-family: quicksand, sans-serif;
  margin: 0;
  padding: 0;
}

.accordion {
  width: 100%;
  max-width: 600px;
  margin: 20px auto;
  display: flex;
  flex-direction: column;
  align-items: center;
}

.title {
  font-size: 2rem;
```

```
  margin-bottom: 2rem;
}

.question {
  width: 100%;

  margin-bottom: 5px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.faq {
  width: 100%;
  border-bottom: 2px solid black;
}

.answer {
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.3s ease-in-out;
}

.faq.active .answer {
  max-height: 200px;
  animation: fade 1s ease-in-out;
}

.faq.active svg {
  transform: rotate(180deg);
}
svg {
  transition: transform 0.3s ease-in-out;
}

@keyframes fade {
```

```
  from {
    opacity: 0;
    transform: translate(-10px);
  }
  to {
    opacity: 1;
    transform: translate(0px);
  }
}
```

```
// list of questions
// each question will have content as answer

// after a title it will have toggle arrow icon
// open or close the content

const faqs = document.querySelectorAll(".faq");

faqs.forEach((faq) => {
  faq.addEventListener("click", () => {
    // Toggle the content visibility
    faq.classList.toggle("active");
  });
});
```



4. Create a Debounced Search Box

Prompt:

Create a search input that logs the typed value after 500ms of no typing (debounce). No API call needed, just log it to console.

> Follow-up: Can you explain what debounce and throttle are and where you'd use them?

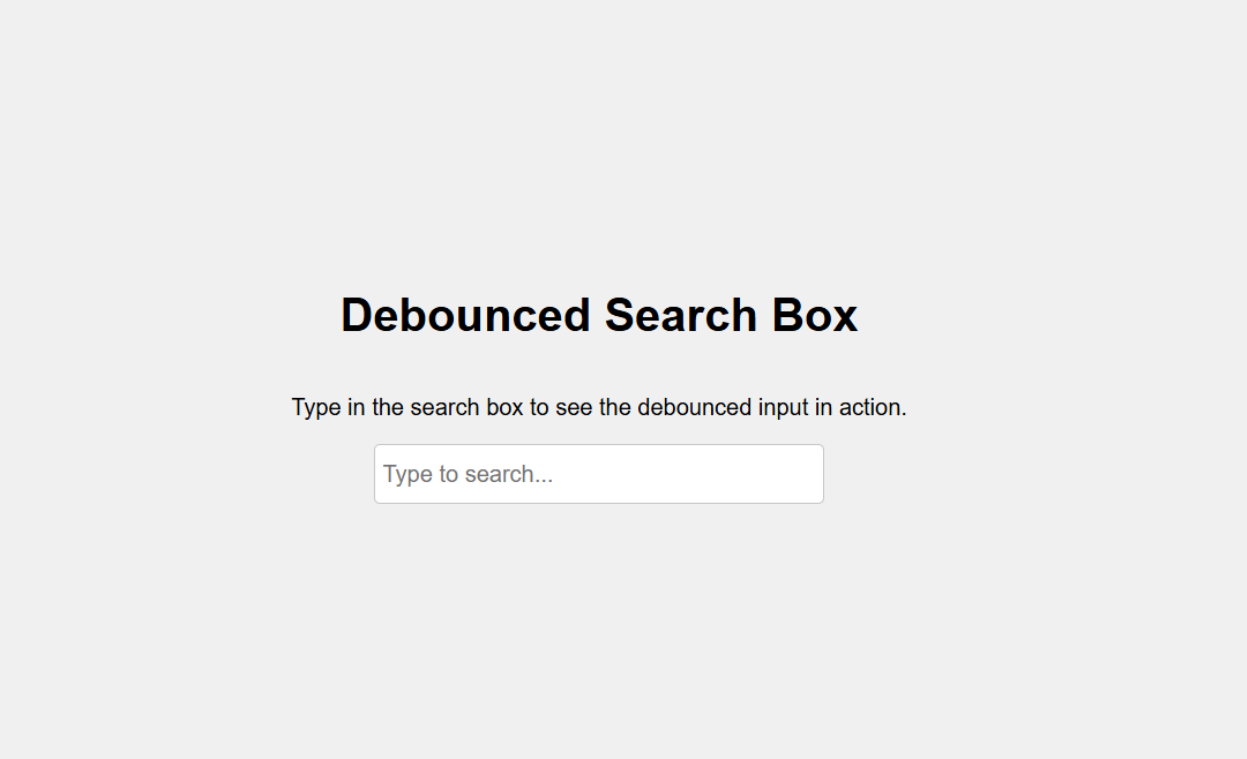
```
<!DOCTYPE html>
<html>
 <link rel="stylesheet" href="search.css" />

 <head> </head>
```

```
  <title>Debounced Search Box</title>
  <body>
    <h1>Debounced Search Box</h1>
    <p>Type in the search box to see the debounced input in action.</p>

    <input type="text" id="searchInput" placeholder="Type to search..." />
    <script src="search.js"></script>
  </body>
</html>
```

```
html,
body {
  height: 100%;
  margin: 0;
}

body {
  font-family: Arial, sans-serif;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  background-color: #f4f4f4;
}

#searchInput {
  width: 300px;
  height: 30px;
  border: 1px solid #ccc;
  padding: 5px;
  font-size: 16px;
  border-radius: 4px;
}
```

```
let input = document.getElementById("searchInput");

function debounce(fn, delay) {
  let timeoutId;
  return function (...args) {
    console.log("testing timeout id", timeoutId);
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

function handleInput(e) {
  console.log("event", e.target.value);
}

input.addEventListener("input", debounce(handleInput, 2000));

function throttle(func, limit) {
  let lastCall = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastCall >= limit) {
      lastCall = now;
      func.apply(this, args);
    }
  };
}

// Explanation of debounce and throttle:
// Debounce is a technique used to limit the rate at which a function is executed.
//ensures that a function is only called after a certain period of inactivity (e.g., 50
//  This is useful for scenarios like search inputs, where you want to avoid making
```

```
// Throttle, on the other hand, ensures that a function is called at most once in a s
// This is useful for scenarios like scrolling or resizing events, where you want to
// of times a function is executed to improve performance and avoid overwhelmir
// In summary, use debounce when you want to wait for a pause in activity befor
// and use throttle when you want to ensure a function is executed at regular inte
```



5. Render Nested Comments with “View More Replies”

Prompt:

Create a nested comment rendering system using vanilla JavaScript.

Each comment may have zero or more replies. If a comment has replies, show a **"View more replies"** button below it. When the button is clicked, the replies should render recursively under that comment. Ensure that replies are rendered **only once per comment** (i.e., repeated clicks should not duplicate replies).

```html
<!DOCTYPE html>
<html>
 <head>
  <title>view comments</title>
  <link rel="stylesheet" href="comments.css" />
 </head>

 <body>
  <h1>Comment section</h1>
  <div id="comments"></div>
 </body>
 <script src="comments.js"></script>
</html>
```

```css
.reply {
 margin-left: 30px;
}
```

```js
const commentsData = [
 {
  id: 1,
  content: "John said he likes burgers",
  replies: [
   {
    id: 12,
    content: "Oh yes I like burgers as well",
    replies: [
     {
      id: 14,
      content: "I like pizza more than burgers",
     },
     {
```

```
          id: 15,
          content: "pizza is good",
        },
        {
          id: 16,
          content: "chocolate is the best",
        },
      ],
    },
    {
      id: 13,
      content: "yeah me too lets go eat together",
      replies: [],
    },
  ],
 },
 {
  id: 2,
  content: "What chocolate is the best in the area",
  replies: [],
 },
];

const comments = document.getElementById("comments");
function renderComments(data) {
 const container = document.createElement("div");

 for (let i = 0; i < data.length; i++) {
  const comment = document.createElement("div");
  const content = document.createElement("p");
  content.innerText = `${data[i].content}`;

  comment.appendChild(content);

  const commentChild = document.createElement("div");
  if (data[i].replies && data[i].replies.length > 0) {
```

```
      const button = document.createElement("button");
      button.textContent = "view more replies";
      commentChild.classList.add("reply");
      button.addEventListener("click", () => {
        console.log("button clicked");
        if (commentChild.childElementCount === 0) {
          commentChild.appendChild(renderComments(data[i].replies));
        }
      });

      comment.appendChild(button);
    }

    comment.appendChild(commentChild);
    container.appendChild(comment);
  }
  return container;
}

comments.appendChild(renderComments(commentsData));
```

Comment section

John said he likes burgers

view more replies

Oh yes I like burgers as well

view more replies

I like pizza more than burgers

pizza is good

chocolate is the best

yeah me too lets go eat together

What chocolate is the best in the area